



FACULDADE DE CIÊNCIAS E TECNOLOGIA

Câmpus Aparecida de Goiânia

Curso de Engenharia de Produção – *Práticas em Engenharia de Produção*

Professor: **Dayvid W P Martins**

Aula de Práticas em Eng. Produção # 05

Banco de Dados para Engenharia de Produção

Aula de Práticas em Eng. Produção # 05



1. Por que BD importa?

Dados como ativo estratégico

2. Relacional: Relacionamentos

1:1, 1:N, N:M e Normalização (1FN–3FN)

3. SQL na Prática

Consultas, JOINS e agregações

4. NoSQL: MongoDB

Composição, referência e agregação

5. Cassandra e Polyglot

Séries temporais e quando usar cada BD

6. Estudo de Caso e Exercícios

Fábrica de componentes automotivos

"Sem dados você é apenas mais uma pessoa com uma opinião."
— W. Edwards Deming



Reflexão!



Dados como ativo estratégico
na produção

1. Por que
BD
importa?

O papel dos dados na Eng. Produção



Todo sistema de informação (ERP, MES, WMS) depende de um **banco de dados**

O engenheiro de produção interage diariamente com dados de:

1. Estoque e matéria-prima
2. Ordens de produção e sequenciamento
3. Indicadores de qualidade (CEP, PPM)
4. Tempos de setup e ciclo

Saber extrair e interpretar dados é competência essencial

Analogia: BD é o "armazém organizado" dos dados da empresa

Dado vs Informação vs Conhecimento



Nível	Exemplo	Significado
Dado bruto	"150 unidades"	Valor sem contexto
Informação	"Estoque Produto A = 150 un, abaixo do ponto de pedido"	Contexto agregado
Conhecimento	"Preciso fazer pedido de compra, lead time = 5 dias"	Ação e decisão

→ O BD armazena dados; as consultas (queries) transformam em informação

Tabelas, relacionamentos e
normalização

2. Banco de Dados Relacional

O que é um BD Relacional?



Modelo proposto por **Edgar Codd** (1970) na IBM

1. **Dados organizados em tabelas** (relações) com linhas e colunas
2. **Cada tabela tem uma chave primária (PK)** — identifica cada registro de forma única
3. **Linguagem padrão:** SQL (Structured Query Language)

Principais SGBDs:

- **PostgreSQL** — open source, robusto, analytics
- **MySQL / MariaDB** — web, SaaS, alta performance em leituras
- Oracle, SQL Server — corporativos

Anatomia de uma Tabela



Exemplo: Tabela **produtos** (indústria de componentes)

id	nome	categoria	preco_unitario	qtd_estoque
1	Parafuso M6	Fixação	0.45	15000
2	Chapa Aço 2mm	Matéria-prima	85.00	320
3	Rolamento 6205	Componente	42.90	890

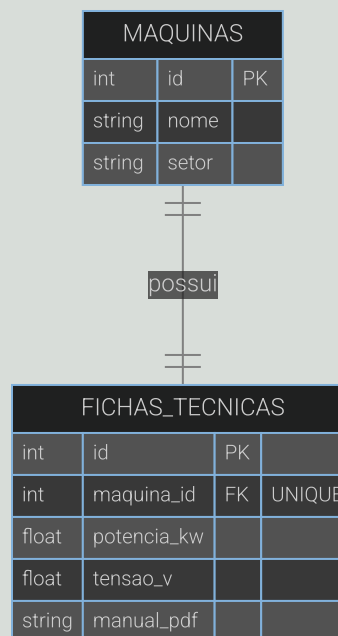
- **Registro (linha):** cada linha é um produto
- **Campo (coluna):** cada coluna é um atributo
- **Chave Primária:** a coluna **id** identifica de forma única cada produto

Relacionamento 1:1 — Um para Um



Cada registro de uma tabela se relaciona com **exatamente um** registro de outra

Exemplo EP — Máquina e Ficha Técnica:



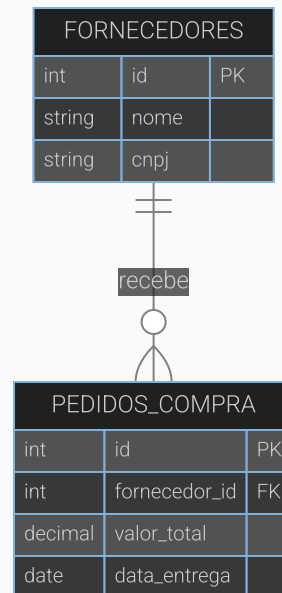
→ Dados **opcionais** ou grandes → tabela separada com FK UNIQUE

Relacionamento 1:N — Um para Muitos



Um registro da tabela A se relaciona com **vários** registros da tabela B

Exemplo EP — Fornecedor e Pedidos de Compra:



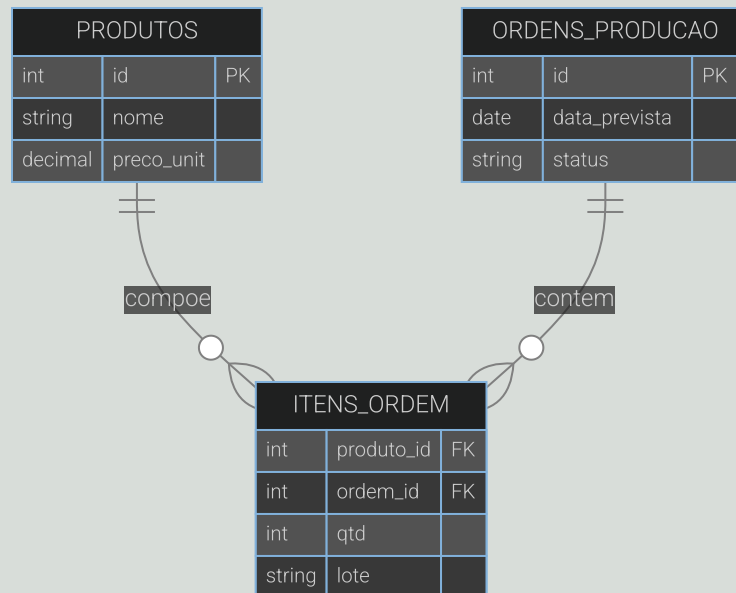
→ É o relacionamento **mais comum** em sistemas industriais (ERP, MES, WMS)

Relacionamento N:M — Muitos para Muitos



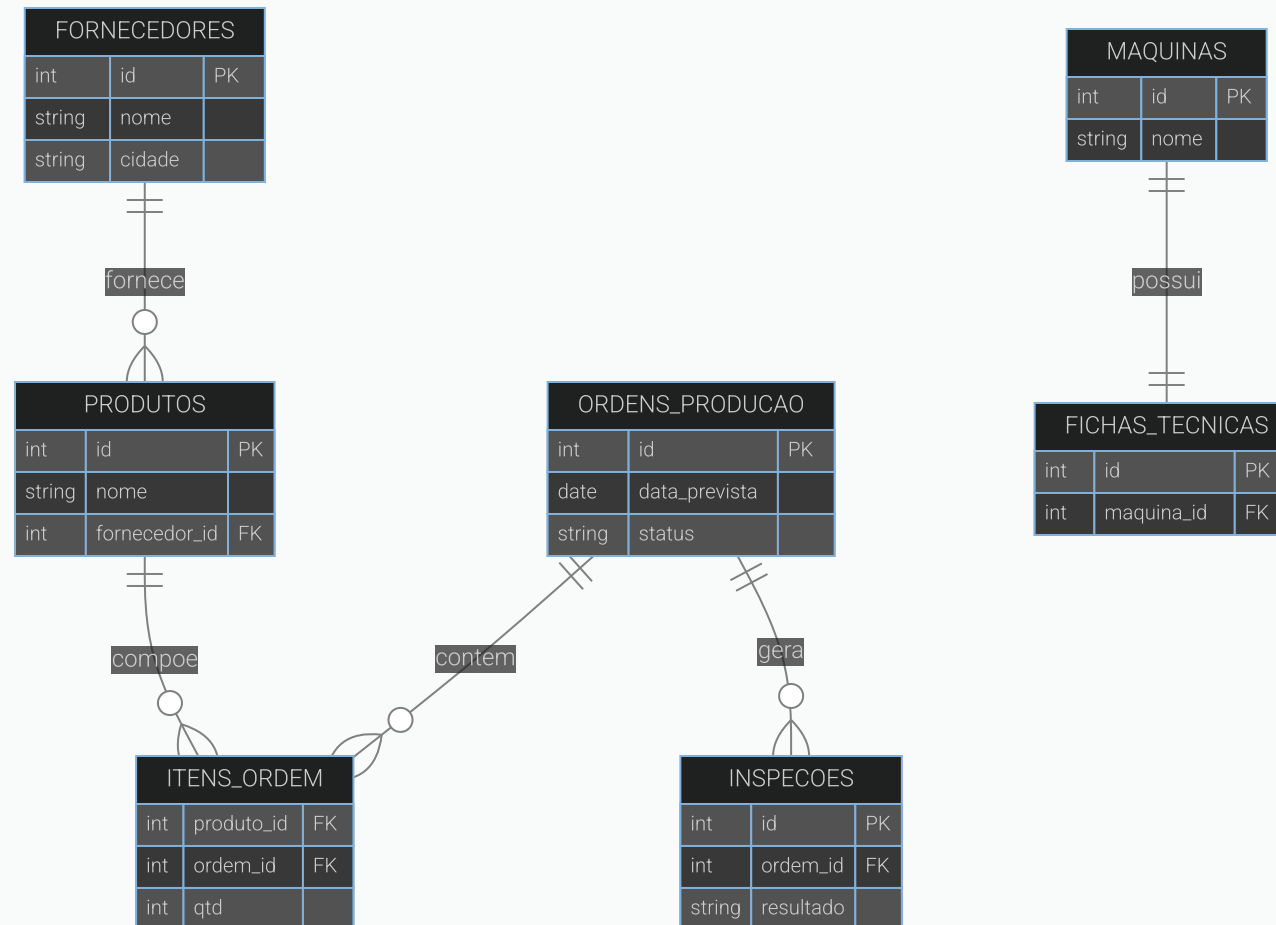
Precisa de **tabela intermediária** para resolver a relação

Exemplo EP — Produtos e Ordens de Produção:



→ A tabela **ITENS_ORDEM** resolve o N:M e guarda dados próprios (quantidade, lote)

Modelo Completo — Fábrica de Componentes (UML)



Chave Estrangeira e Integridade Referencial



Chave Estrangeira (FK): campo que referencia a PK de outra tabela

```
CREATE TABLE pedidos_compra (  
    id SERIAL PRIMARY KEY,  
    fornecedor_id INTEGER NOT NULL  
        REFERENCES fornecedores(id), -- FK  
    valor_total DECIMAL(10,2),  
    data_entrega DATE  
);
```

Integridade referencial — o BD **impede** que:

- Um pedido referencie um fornecedor inexistente
- Um fornecedor seja deletado se tiver pedidos vinculados

→ Garante **consistência** dos dados no chão de fábrica



Objetivo: **eliminar redundância** e anomalias de inserção/atualização/exclusão

Por que normalizar? Imagine um sistema de produção onde o nome do fornecedor está repetido em 500 pedidos. Se ele mudar de razão social, você precisa atualizar 500 linhas.

Formas Normais:

- **1FN:** cada célula com **valor atômico** (sem listas ou grupos repetitivos)
- **2FN:** todo atributo depende da **chave primária completa** (sem dependências parciais)
- **3FN:** nenhum atributo não-chave depende de **outro atributo não-chave**

1FN — Primeira Forma Normal



Problema: campo com múltiplos valores (violação da atomicidade)

ordem_id	produtos	quantidades
1001	Eixo A, Rolamento 6205	500, 200

→ **Violação da 1FN:** "produtos" e "quantidades" contêm listas!

Solução (1FN): separar em linhas individuais

ordem_id	produto	quantidade
1001	Eixo A	500
1001	Rolamento 6205	200

→ Cada célula agora contém um **valor atômico** ✓

2FN — Segunda Forma Normal



Problema: atributo depende apenas de parte da chave composta

ordem_id	produto_id	nome_produto	qtd
1001	3	Rolamento 6205	200
1002	3	Rolamento 6205	150

PK composta = (ordem_id, produto_id). Mas **nome_produto** depende **só** de **produto_id**

→ **Violação da 2FN:** dependência parcial!

Solução: separar em duas tabelas

- **itens_ordem:** ordem_id, produto_id, qtd
- **produtos:** produto_id, nome_produto

3FN — Terceira Forma Normal



Problema: atributo não-chave depende de outro atributo não-chave

fornecedor_id	nome	cidade	estado
1	Aços Goiás	Goiânia	GO
2	Metal Sul	Curitiba	PR

estado depende de **cidade**, não de **fornecedor_id** diretamente

→ **Violação da 3FN:** dependência transitiva!

Solução: separar localidades

- **fornecedores:** id, nome, cidade_id
- **cidades:** id, nome, estado

→ Na prática de EP: elimina inconsistências quando a fábrica tem centenas de fornecedores cadastrados

Normalização — Visão Completa (Cenário EP)



ANTES (desnormalizado) — planilha do chão de fábrica:

ordem	produto	fornec	cidade_fornec	qtd	preco_unit
1001	Eixo A	Aços Goiás	Goiânia	500	12.50
1002	Eixo A	Aços Goiás	Goiânia	300	12.50

DEPOIS (3FN) — 4 tabelas normalizadas:

- **ordens_producao:** id, data_prevista, status
- **itens_ordem:** ordem_id (FK), produto_id (FK), qtd
- **produtos:** id, nome, preco_unit, fornecedor_id (FK)
- **fornecedores:** id, nome, cidade

→ "Aços Goiás / Goiânia / 12.50" agora aparece **uma única vez** no BD ✓

Consultas, filtros, agregações
e JOINS

3. SQL na Prática

Principais Comandos SQL



Comando	Função	Exemplo
SELECT	Consultar dados	SELECT nome FROM produtos
INSERT	Inserir registros	INSERT INTO produtos ...
UPDATE	Atualizar dados	UPDATE produtos SET ...
DELETE	Remover registros	DELETE FROM produtos ...
WHERE	Filtros condicionais	WHERE qtd < 500
ORDER BY	Ordenação	ORDER BY nome ASC
GROUP BY	Agrupamento	GROUP BY categoria



Exemplo prático: Produtos com estoque baixo

```
-- Produtos com estoque abaixo de 500 unidades  
SELECT nome, qtd_estoque  
FROM produtos  
WHERE qtd_estoque < 500  
ORDER BY qtd_estoque ASC;
```

Resultado:

nome	qtd_estoque
Chapa Aço 2mm	320
Rolamento 6205	890

JOINS — Cruzando Tabelas



Exemplo: Pedidos de compra com dados do fornecedor

```
SELECT p.numero_pedido, f.nome_fornecedor,  
       p.valor_total, p.data_entrega  
FROM pedidos_compra p  
INNER JOIN fornecedores f ON p.fornecedor_id = f.id  
WHERE p.status = 'pendente';
```

Tipos de JOIN:

- **INNER JOIN** → apenas registros com correspondência
- **LEFT JOIN** → todos da esquerda + correspondências
- **RIGHT JOIN** → todos da direita + correspondências

Agregações (Cenário de Produção)



Produção total por linha de montagem

```
SELECT linha_montagem,  
       COUNT(*) AS total_ordens,  
       SUM(quantidade) AS total_pecas,  
       AVG(tempo_ciclo) AS tempo_medio_ciclo  
FROM ordens_producao  
WHERE data_inicio >= '2026-03-01'  
GROUP BY linha_montagem  
ORDER BY total_pecas DESC;
```

→ Resultado mostra produtividade por linha em 1 mês



Produtos que nunca tiveram defeito registrado

```
SELECT p.nome
FROM produtos p
WHERE p.id NOT IN (
    SELECT DISTINCT produto_id
    FROM registros_qualidade
    WHERE tipo = 'defeito'
);
```

Técnicas avançadas:

- **IN / NOT IN** → verificar existência em sublista
- **EXISTS** → validar condicional (mais eficiente)
- **Subqueries correlacionadas** → referências cruzadas entre tabelas

PostgreSQL vs MySQL vs MariaDB



Característica	PostgreSQL	MySQL	MariaDB
Licença	Open Source	Oracle/GPL	GPL (fork MySQL)
Conformidade SQL	Alta	Moderada	Moderada
Tipos avançados	JSON, Array, GIS	Padrão	Padrão + extras
Uso típico	Analytics, GIS	Web, SaaS	Web, substituto MySQL
Performance	Consultas complexas	Leituras simples	Leituras simples

Recomendação EP: PostgreSQL para sistemas robustos, MySQL para aplicações web

MongoDB, Cassandra e quando
usar

4. Banco de Dados NoSQL

O que é NoSQL?



"Not Only SQL" — complementa, não substitui o relacional

Surgiu para lidar com:

1. **Volume massivo de dados** (Big Data)
2. **Estruturas flexíveis** e variáveis
3. **Alta disponibilidade** e escalabilidade

Tipos de bancos NoSQL:

- **Documentos** → MongoDB
- **Chave-Valor** → Redis
- **Colunar** → Cassandra
- **Grafos** → Neo4j

MongoDB — Banco de Documentos



Armazena dados em documentos **JSON/BSON** (sem esquema rígido)

Conceito-chave: em vez de tabelas com linhas e colunas, temos **coleções** de **documentos**

Exemplo EP — Registro de máquina com sensores IoT:

```
{
  "maquina_id": "CNC-03",
  "nome": "Torno CNC Romi GL 240",
  "setor": "Usinagem",
  "sensores": [
    {"tipo": "temperatura", "valor": 72.5, "unidade": "°C"},
    {"tipo": "vibração", "valor": 0.8, "unidade": "mm/s"}
  ],
  "ultima_manutencao": "2026-02-15"
}
```

→ Cada máquina pode ter **sensores diferentes** — schema flexível

MongoDB — Composição de Documentos



No MongoDB, usamos **documentos embutidos (embedded)** para representar composições

Composição = um objeto que **não existe sem o pai** (relação "parte de")

Exemplo EP — Ordem de Produção com seus itens:

```
{
  "ordem_id": "OP-2026-0342",
  "data_prevista": "2026-03-10",
  "linha_montagem": "Linha 02",
  "status": "em_andamento",
  "itens": [
    {"produto": "Eixo A", "qtd": 500, "tempo_ciclo_s": 45},
    {"produto": "Bucha B", "qtd": 1000, "tempo_ciclo_s": 12}
  ],
  "responsavel": {"nome": "Carlos", "matricula": "F-0451"}
}
```

MongoDB — Referência vs Composição



Quando embutir (composição) e quando referenciar?

Critério	Embutir (Composição)	Referenciar
Relação	"parte de" (1:poucos)	entidade independente
Acesso	lidos juntos sempre	acessados separadamente
Tamanho	subdocumento pequeno	dados grandes ou compartilhados
Exemplo EP	Itens dentro da OP	Fornecedor referenciado pela OP

Referência (como FK no relacional):

```
{ "ordem_id": "OP-2026-0342",  
  "fornecedor_id": ObjectId("65f1a..."),  
  "itens": [ ... ] }
```

→ `fornecedor_id` aponta para outro documento na coleção `fornecedores`

MongoDB — Aggregation Pipeline



O **pipeline de agregação** é o equivalente do **GROUP BY + JOIN** do SQL

Exemplo EP — Produção total por linha de montagem (último mês):

```
db.ordens_producao.aggregate([
  { $match: {
    data_prevista: { $gte: ISODate("2026-03-01") }
  } },
  { $unwind: "$itens" },
  { $group: {
    _id: "$linha_montagem",
    total_ordens: { $sum: 1 },
    total_pecas: { $sum: "$itens.qtd" },
    tempo_medio: { $avg: "$itens.tempo_ciclo_s" }
  } },
  { $sort: { total_pecas: -1 } }
])
```



Exemplo EP — Taxa de defeitos por tipo de inspeção:

```
db.inspecoes.aggregate([
  { $group: {
    _id: "$tipo_inspecao",
    total: { $sum: 1 },
    reprovados: { $sum: {
      $cond: [{ $eq: ["$resultado", "reprovado"] }, 1, 0]
    }}
  }},
  { $addFields: { taxa defeito: { $round: [
    { $multiply: [{ $divide: ["$reprovados", "$total"] }, 100] }, 2
  ]}}}
])
```

→ Taxa de defeito sobre documentos de inspeção — sem JOINS

Cassandra — Banco Colunar Distribuído



Projetado para alta disponibilidade e escalabilidade horizontal

1. Sem ponto único de falha — todos os nós são iguais
2. Replicação automática entre data centers
3. Escala linear com adição de servidores

Ideal para Eng. Produção:

- Séries temporais — leituras de sensores a cada segundo
- Logs de produção — milhões de registros por dia
- Telemetria distribuída — múltiplas fábricas em cidades diferentes

Usuários: Netflix, Apple, empresas com operações globais de manufatura

SQL vs NoSQL — Quando usar cada um?



Critério	SQL (Relacional)	NoSQL (MongoDB/Cassandra)
Estrutura	Fixa, esquema rígido	Flexível, schema-less
Transações	ACID (confiável)	BASE (eventualmente consistente)
Escala	Vertical (hardware maior)	Horizontal (mais servidores)
Relacionamentos	JOINS nativos, FKs	Documentos embutidos, referências
Exemplo EP	ERP, estoque, financeiro	IoT, logs, sensores, catálogos

→ Estratégia moderna: usar **ambos** (**polyglot persistence**)

PostgreSQL para ordens e estoque + MongoDB para sensores IoT + Cassandra para telemetria

Sistema de controle de
produção

5. Estudo de Caso

Cenário: Fábrica de Componentes Automotivos



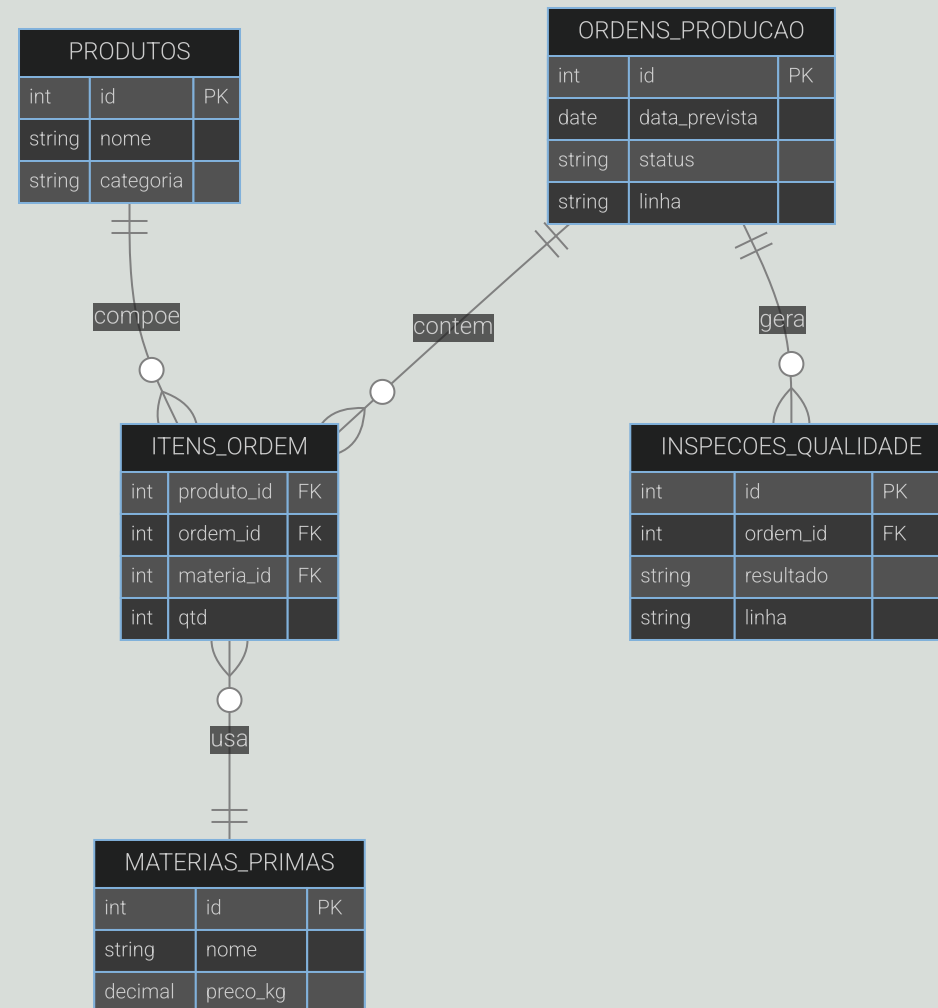
Estrutura da fábrica:

- 3 linhas de montagem
- Produção de eixos, rolamentos, buchas
- Controle de qualidade em tempo real

O sistema precisa controlar:

1. Cadastro de produtos e matéria-prima
2. Ordens de produção e sequenciamento
3. Inspeção de qualidade por lote
4. Rastreabilidade completa

Modelo Relacional do Estudo de Caso



Queries do Dia-a-Dia do Engenheiro



Query 1: Ordens atrasadas

```
SELECT op.id, p.nome, op.data_prevista, op.status
FROM ordens_producao op
JOIN produtos p ON op.produto_id = p.id
WHERE op.data_prevista < CURRENT_DATE
      AND op.status != 'concluida';
```

Query 2: Taxa de defeitos por linha

```
SELECT linha,
       ROUND(100.0 * SUM(CASE WHEN resultado='reprovado'
                              THEN 1 ELSE 0 END) / COUNT(*), 2) AS taxa_defeito_pct
FROM inspecoes_qualidade
GROUP BY linha;
```


Prática com queries reais

6. Exercícios

Exercício Proposto



Dadas as tabelas: **produtos**, **ordens_producao**, **fornecedores**, **materias_primas**

Questões:

1. Liste todos os fornecedores que entregaram matéria-prima no último mês
2. Calcule o custo total de matéria-prima por ordem de produção
3. Identifique o produto com maior número de ordens de produção

Dica: Combine **JOIN**, **WHERE**, **GROUP BY** e agregações (**SUM**, **COUNT**)



- ✓ **Banco de dados** é a espinha dorsal de qualquer sistema industrial
- ✓ **SQL** é a linguagem universal para extrair valor dos dados
- ✓ **Normalização** evita redundância e garante integridade
- ✓ **NoSQL** complementa o relacional para alto volume e flexibilidade
- ✓ O engenheiro de produção que **domina dados** toma decisões melhores

Próximos passos:

- Praticar SQL em PostgreSQL / MySQL
- Explorar MongoDB para dados não estruturados
- Integrar BD em projetos de automação industrial



Livros:

- STAIR, R.M.; REYNOLDS, G.W. *Princípios de Sistemas de Informação*. Cengage, 2015.
- DATE, C.J. *Introdução a Sistemas de Bancos de Dados*. 8ª ed. Campus, 2004.
- SILBERSCHATZ, A. et al. *Sistema de Banco de Dados*. 7ª ed. GEN LTC, 2020.

Documentações:

- PostgreSQL: postgresql.org/docs
- MySQL: dev.mysql.com/doc
- MongoDB: mongodb.com/docs/manual
- Cassandra: cassandra.apache.org/doc

Ferramentas: DBeaver, pgAdmin, MySQL Workbench



Obrigado! ≤3

